

Kingdom of Saudi Arabia  
Ministry of Education  
Qassim University  
College of Computer  
Information Technology Dept.



المملكة العربية السعودية  
وزارة التعليم  
جامعة القصيم  
كلية الحاسب  
قسم تقنية المعلومات

# TrapPot – AI-Powered IoT Honeypot System

*Students:*

Sultan Almansour 431108175

Rayan Alsamani 431108144

Faris Aljamaan 431107708

Sulaiman Albati 421107931

*Supervisor:*

**Dr. Tarek Moulahi**

*A project report submitted in partial fulfillment of the requirements  
for B.Sc. degree in Information Technology.*

*Qassim-Saudi Arabia*

*1446/1447 (2025/2026)*

## Certificate

This is to certify that this project report, entitled *TrapPot – AI-Powered IoT Honeypot System*, has been prepared under my supervision and guidance. The report is approved for submission for evaluation as part of the requirements for the Bachelor of Science degree in Information Technology.

**Dr. Tarek Moulahi**

## Dedication

To our families, whose support, patience, and encouragement helped us complete this work; to our supervisor, Dr. Tarek Moulahi, for his guidance throughout this project; and to Qassim University for providing an environment of learning and innovation, we dedicate this project.

**Sultan Almansour, Rayan Alsamani, Faris Aljamaan, and Sulaiman Albati**

## Acknowledgement

We would like to express our sincere gratitude to our supervisor, Dr. Tarek Moulahi, for his guidance, feedback, and support throughout the development of this project. We also thank the Department of Information Technology at Qassim University for providing the academic environment that made this work possible.

We are grateful to our families for their patience and encouragement during the project period. Their support helped us complete the research, implementation, and documentation stages of TrapPot.

**Sultan Almansour, Rayan Alsamani, Faris Aljamaan, and Sulaiman Albati**

## Abstract

The growth of Internet of Things (IoT) infrastructure in Saudi Arabia, especially under Vision 2030 smart-city initiatives, increases the need for practical systems that can observe and analyze attacks against exposed devices. Many IoT devices still rely on weak credentials, limited update mechanisms, and remote access services such as SSH and Telnet. These conditions make IoT environments attractive targets for botnets and automated attacks.

This project presents TrapPot, an AI-powered IoT honeypot system that combines Cowrie, Zeek, a Random Forest classifier, and the ELK Stack. Cowrie emulates SSH and Telnet services in an isolated Docker-based environment, while Zeek converts network traffic into structured connection logs. The AI detector uses twelve Zeek-compatible features from the trained model pipeline to classify traffic into five classes: brute-force, scanning, malware download, command execution, and normal traffic. Logstash, Elasticsearch, and Kibana support log processing, indexing, and dashboard-based monitoring.

The Random Forest model was trained using the IoT-23 dataset and achieved an accuracy of approximately 99.59% on the prepared dataset. The implemented system was also tested locally through Docker Compose, confirming that Cowrie captured sessions, Zeek generated `conn.log` and `ssh.log`, the AI detector produced live classification output, and Kibana displayed the collected data through the TrapPot dashboard. TrapPot therefore provides a working proof of concept for IoT honeypot monitoring, network-based classification, and security visualization in a controlled academic environment.

**Keywords:** TrapPot, Internet of Things (IoT), Honeypot, Random Forest (RF), Cybersecurity, Artificial Intelligence (AI).

## Table of Contents

|                                                                                    |           |
|------------------------------------------------------------------------------------|-----------|
| Certificate . . . . .                                                              | i         |
| Dedication . . . . .                                                               | ii        |
| Acknowledgement . . . . .                                                          | iii       |
| Abstract . . . . .                                                                 | iv        |
| <b>Abbreviations</b>                                                               | <b>xi</b> |
| <b>1 INTRODUCTION</b>                                                              | <b>1</b>  |
| 1.1 Challenges . . . . .                                                           | 2         |
| 1.2 Study Scope . . . . .                                                          | 3         |
| 1.3 Goals and Objectives . . . . .                                                 | 3         |
| 1.4 Report Arrangement . . . . .                                                   | 4         |
| <b>2 LITERATURE REVIEW</b>                                                         | <b>5</b>  |
| 2.1 Thematic Analysis of IoT Security Challenges and Technical Foundations . . . . | 6         |
| 2.1.1 Challenges in Project Domain . . . . .                                       | 6         |
| 2.1.1.1 IoT Security Vulnerabilities and Threat Landscape . . . . .                | 6         |
| 2.1.1.2 Honeypot Deployment and Isolation Challenges . . . . .                     | 6         |
| 2.1.1.3 Machine Learning Classification Challenges . . . . .                       | 7         |
| 2.1.1.4 Data Collection and Preprocessing Challenges . . . . .                     | 7         |
| 2.1.2 Related Work . . . . .                                                       | 7         |
| 2.1.3 Technical Background . . . . .                                               | 9         |
| 2.1.3.1 Honeypots . . . . .                                                        | 9         |
| 2.1.3.2 Cowrie Honeypot . . . . .                                                  | 9         |
| 2.1.3.3 SSH and Telnet Protocols . . . . .                                         | 9         |
| 2.1.3.4 ELK Stack . . . . .                                                        | 9         |
| 2.1.3.5 Random Forest Algorithm . . . . .                                          | 10        |

---

|          |                                                                                |           |
|----------|--------------------------------------------------------------------------------|-----------|
| 2.1.3.6  | Zeek Network Security Monitor . . . . .                                        | 10        |
| 2.1.3.7  | Supervised Learning . . . . .                                                  | 10        |
| 2.1.3.8  | Feature Extraction . . . . .                                                   | 10        |
| 2.1.3.9  | Classification Metrics . . . . .                                               | 10        |
| 2.1.3.10 | Attack Types . . . . .                                                         | 11        |
| 2.2      | Chronological Evolution and Progress of IoT Threat Detection Systems . . . . . | 11        |
| 2.2.1    | Related Work . . . . .                                                         | 11        |
| <b>3</b> | <b>METHODOLOGY</b>                                                             | <b>13</b> |
| 3.1      | Research Design . . . . .                                                      | 14        |
| 3.1.1    | Research Design Type: Case Study . . . . .                                     | 14        |
| 3.1.2    | Research Approach: Quantitative . . . . .                                      | 15        |
| 3.2      | Data Collection . . . . .                                                      | 15        |
| 3.2.1    | Pre-Implementation Data Collection . . . . .                                   | 15        |
| 3.2.2    | Post-Implementation Data Collection . . . . .                                  | 16        |
| 3.3      | Data Analysis . . . . .                                                        | 17        |
| 3.3.1    | Data Acquisition and Preprocessing Pipeline . . . . .                          | 17        |
| 3.3.2    | Feature Engineering and Selection . . . . .                                    | 18        |
| 3.3.3    | Machine Learning Model Training and Optimization . . . . .                     | 18        |
| 3.4      | Validation Plan . . . . .                                                      | 19        |
| 3.4.1    | Classification Performance Metrics . . . . .                                   | 19        |
| 3.4.2    | Validation Dataset and Testing Method . . . . .                                | 19        |
| 3.4.3    | Operational Success Criteria . . . . .                                         | 20        |
| 3.4.4    | Results Presentation . . . . .                                                 | 20        |
| 3.5      | Research Ethics . . . . .                                                      | 20        |
| 3.5.1    | Network Isolation and Containment . . . . .                                    | 20        |
| 3.5.2    | Data Handling and Anonymization . . . . .                                      | 21        |
| 3.5.3    | Regulatory Compliance and Log Management . . . . .                             | 21        |
| <b>4</b> | <b>IMPLEMENTATION AND TESTING</b>                                              | <b>22</b> |
| 4.1      | Development Environment and Tools . . . . .                                    | 23        |
| 4.1.1    | Primary Development Platform . . . . .                                         | 23        |
| 4.1.2    | Containerization and Orchestration . . . . .                                   | 23        |

---

---

|          |                                                            |           |
|----------|------------------------------------------------------------|-----------|
| 4.1.3    | Data Processing and Visualization Tools . . . . .          | 24        |
| 4.2      | System Architecture . . . . .                              | 24        |
| 4.2.1    | High-Level Architecture Overview . . . . .                 | 24        |
| 4.2.2    | Component Interaction and Data Flow . . . . .              | 25        |
| 4.2.3    | Network Topology and Isolation . . . . .                   | 25        |
| 4.3      | Component Implementation . . . . .                         | 26        |
| 4.3.1    | Cowrie Honeypot Deployment . . . . .                       | 26        |
| 4.3.2    | Zeek Sensor Deployment . . . . .                           | 26        |
| 4.3.3    | Machine Learning Module Implementation . . . . .           | 26        |
| 4.3.4    | ELK Stack Configuration . . . . .                          | 27        |
| 4.4      | Feature Extraction and Classification Algorithms . . . . . | 28        |
| 4.4.1    | Feature Extraction Algorithm . . . . .                     | 28        |
| 4.4.2    | Random Forest Training Algorithm . . . . .                 | 29        |
| 4.4.3    | Real-Time Classification Algorithm . . . . .               | 29        |
| 4.5      | System Integration and Deployment . . . . .                | 30        |
| 4.5.1    | Deployment Verification . . . . .                          | 30        |
| 4.6      | Testing Strategy . . . . .                                 | 31        |
| 4.6.1    | Unit Testing . . . . .                                     | 31        |
| 4.6.2    | Integration Testing . . . . .                              | 32        |
| 4.6.3    | Performance Testing . . . . .                              | 32        |
| 4.6.4    | Security Testing . . . . .                                 | 32        |
| 4.6.5    | Classifier Accuracy Validation . . . . .                   | 32        |
| <b>5</b> | <b>RESULTS AND DISCUSSIONS</b>                             | <b>34</b> |
| 5.1      | Analysis Strategy . . . . .                                | 35        |
| 5.2      | Analysis Result . . . . .                                  | 36        |
| 5.3      | Major Findings . . . . .                                   | 37        |
| 5.4      | Discussion . . . . .                                       | 38        |
| <b>6</b> | <b>CONCLUSIONS AND FUTURE WORK</b>                         | <b>40</b> |
| 6.1      | Discussion of Results . . . . .                            | 41        |
| 6.1.1    | Contributions . . . . .                                    | 42        |
| 6.2      | Limitations . . . . .                                      | 42        |

---



|                                        |                                                        |           |
|----------------------------------------|--------------------------------------------------------|-----------|
| 6.3                                    | Future Work . . . . .                                  | 43        |
| 6.3.1                                  | Testing with Realistic Attack Traffic . . . . .        | 43        |
| 6.3.2                                  | Combining Zeek and Cowrie Features . . . . .           | 43        |
| 6.3.3                                  | Indexing AI Results in Elasticsearch . . . . .         | 44        |
| 6.3.4                                  | Exploring Additional Machine Learning Models . . . . . | 44        |
| 6.3.5                                  | Improving Deployment Security . . . . .                | 44        |
| 6.4                                    | Conclusion . . . . .                                   | 44        |
| <b>Appendix A Project Installation</b> |                                                        | <b>46</b> |
| <b>References</b>                      |                                                        | <b>47</b> |

## List of Figures

|     |                                                    |    |
|-----|----------------------------------------------------|----|
| 4.1 | TrapPot system architecture and data flow. . . . . | 24 |
|-----|----------------------------------------------------|----|

## List of Tables

|     |                                                  |    |
|-----|--------------------------------------------------|----|
| 5.1 | TrapPot Component Verification Results . . . . . | 37 |
| 5.2 | Random Forest Model Accuracy Result . . . . .    | 37 |

# Abbreviations

|     |                                     |
|-----|-------------------------------------|
| AI  | Artificial Intelligence             |
| ELK | Elasticsearch, Logstash, and Kibana |
| IDS | Intrusion Detection System          |
| IoT | Internet of Things                  |
| ML  | Machine Learning                    |
| RF  | Random Forest                       |
| SSH | Secure Shell                        |
| TCP | Transmission Control Protocol       |

## Chapter 1

# INTRODUCTION

# INTRODUCTION

Saudi Arabia is adopting Internet of Things (IoT) devices in smart-city projects and other digital transformation initiatives under Vision 2030. These devices expand connectivity, but many of them remain exposed because of weak credentials, limited patching, and insecure remote access services. The 2016 Mirai botnet demonstrated the scale of this risk by compromising more than 600,000 IoT devices worldwide and using them to cause major internet outages [1]. This incident shows that insecure IoT devices are not only a technical issue, but also a threat to companies, infrastructure, and public digital services.

Honeypots provide a controlled way to attract attackers and study their behavior without exposing production systems. At the same time, artificial intelligence can help classify attack patterns and reduce the amount of manual analysis required from security teams. TrapPot combines these two ideas in one system. It uses a Cowrie-based SSH and Telnet honeypot to capture attacker interaction, Zeek to summarize network traffic, a Random Forest model to classify traffic behavior, and the ELK Stack to store and visualize the collected evidence. The goal is to support practical IoT threat monitoring and make attack behavior easier to understand in an academic and local cybersecurity context.

## 1.1 Challenges

Several challenges appear when developing an IoT honeypot system. First, the project requires knowledge of cybersecurity, networking, containerization, log management, and machine learning, which makes it technically demanding for an undergraduate team. Second, IoT honeypot projects have fewer practical tutorials and case studies than more common IT topics, so the team had to rely on academic papers, official documentation, and direct testing. Third, a honeypot must be isolated carefully. If it is connected to the wrong network or configured without containment, attackers could misuse it as an entry point or as a platform for further activity. Finally, the system must handle continuous log collection, storage, feature extraction, and classification without losing the relationship between attacker behavior and network traffic.

TrapPot addresses these challenges by using established open-source tools, a Docker-based deployment, and strict lab-network assumptions. Cowrie captures attacker interaction, Zeek produces structured network logs, the Random Forest detector classifies traffic, and ELK provides centralized monitoring. This design reduces the amount of custom infrastructure needed

while still allowing the project to study real attack behavior in a controlled way.

## 1.2 Study Scope

This research project aims to create an AI-based honeypot system that emulates Telnet and SSH services to monitor IoT-targeted attacks inside a standalone network segment. The scope is limited to Random Forest classification for five classes: brute-force, scanning, malware download, command execution, and normal traffic. The study also keeps the original 30-day collection objective and the minimum 99% classification accuracy target. The implemented system operates as a single-node proof of concept with network isolation to avoid compromising production systems. Advanced protocols such as HTTP, MQTT, and CoAP, deep learning methods, distributed deployments, and production-ready hardening are outside the scope of this phase and are reserved for future work.

Random Forest has already shown high accuracy and low latency for classifying honeypot-captured IoT botnet attacks, making it a suitable choice for TrapPot's classifier [2]. Telnet and SSH services are also primary targets in Mirai-like IoT botnet activity, which motivates the focus on these protocols and the selected attack types [3, 4].

## 1.3 Goals and Objectives

The project aims to improve IoT security readiness by observing attacker behavior, analyzing network traffic, and classifying suspicious activity. The main objectives are:

- **Understanding IoT Attack Behavior:** Investigate attacker interactions to understand how IoT-like systems are discovered, probed, and exploited. This includes analyzing commands, login attempts, and traffic patterns produced during attack sessions.
- **Improving IoT Attack Classification:** Develop an AI-based classifier that can categorize brute-force attempts, scanning behavior, normal traffic, malware-related activity, and command execution. A core objective is to achieve at least 99 percent classification accuracy on the prepared dataset.
- **Supporting Proactive Security Measures:** Provide earlier visibility into malicious behavior so that security teams can apply preventive measures before damage occurs.

- **Enhancing IoT Threat Awareness:** Present captured activity in a structured way so researchers, administrators, and organizations can understand common attack methods and trends.
- **Reducing the Attack Surface of IoT Environments:** Use findings from observed attack behavior to guide practical recommendations, such as improving access controls and reducing unnecessary service exposure.

Together, these objectives support better decision-making and stronger security practices for IoT environments facing changing cyberattack patterns.

## 1.4 Report Arrangement

This report is organized into six chapters:

- **Chapter 1: Introduction** presents the IoT security problem, project motivation, objectives, and scope.
- **Chapter 2: Literature Review** reviews existing work on IoT honeypots and machine learning for attack classification, and provides technical background for Cowrie, Random Forest, Zeek, and the ELK Stack.
- **Chapter 3: Methodology** explains the research design, data sources, feature engineering, validation plan, and ethical considerations.
- **Chapter 4: Implementation and Testing** documents the implemented Docker-based TrapPot system and the verification of its main components.
- **Chapter 5: Results and Discussions** presents the implementation results, model accuracy, dashboard verification, and system limitations.
- **Chapter 6: Conclusions and Future Work** summarizes the project outcomes and suggests future improvements.



## Chapter 2

# LITERATURE REVIEW

# LITERATURE REVIEW

This chapter reviews existing work related to IoT security, with emphasis on cyberattack classification, intrusion detection systems, honeypots, AI-based security analysis, and log visualization. The review connects these areas to the design of TrapPot and explains why the project combines Cowrie, Zeek, Random Forest classification, and the ELK Stack.

## 2.1 Thematic Analysis of IoT Security Challenges and Technical Foundations

### 2.1.1 Challenges in Project Domain

#### 2.1.1.1 IoT Security Vulnerabilities and Threat Landscape

The Internet of Things faces major security risks because many devices have limited resources, weak authentication, and irregular security updates. Mahmood and colleagues reported that encryption protection was missing from 55% of IoT devices, while authentication issues affected the device, network, and application layers [5]. The growth of IoT botnets shows the practical impact of these weaknesses, as malware families such as Mirai, Gafgyt, and Mozi continue to use default passwords and exposed services to compromise devices [6]. Islam and colleagues analyzed more than 11,000 IoT code examples and found that 39.58% included real Common Vulnerabilities and Exposures instances, with 28.99% related to denial-of-service errors that threaten network-connected IoT systems [7]. These findings support TrapPot's focus on exposed remote access services and automated attack behavior.

#### 2.1.1.2 Honeypot Deployment and Isolation Challenges

Deploying honeypots for IoT security research creates both technical and operational challenges. Adaptive honeypot research has shown that fixed honeypot designs can struggle in IoT environments because they must imitate different devices while preserving realism and safety [8]. Network isolation is also critical because a poorly contained honeypot can become a path into production networks or a platform for attacking other systems [9]. Attackers may also try to detect decoy systems through behavioral analysis and timing fingerprints, so honeypots must provide believable interaction while keeping strict containment [10]. For an educational project,

these constraints make a medium-interaction SSH and Telnet honeypot a practical balance between realism, safety, and implementation effort.

#### **2.1.1.3 Machine Learning Classification Challenges**

Machine learning can help classify honeypot activity, but the approach introduces several challenges. A 2023 survey on machine-learning-based intrusion detection for IoT systems identified dataset imbalance as a major problem because benign traffic often appears in much larger quantities than rare attack classes [11]. Several IoT studies report Random Forest accuracy above 99% for multi-class attack detection, but models can still struggle when feature sets become large or when deployment requires real-time operation on limited resources [12]. Feature engineering therefore requires domain knowledge. Weak or irrelevant features can reduce classification quality and increase processing cost [13]. Model performance also depends on the quality and diversity of training data, especially when new botnet variants or zero-day behaviors differ from the examples seen during training.

#### **2.1.1.4 Data Collection and Preprocessing Challenges**

Honeypots can produce large volumes of interaction data that must be stored, cleaned, labeled, and transformed before they can support classification. Rodriguez-Gomez and colleagues noted that manual attack labeling is time-consuming, inconsistent, and difficult when attack patterns are ambiguous [14]. Public IoT security datasets are useful, but they differ in collection methods, feature structures, and attack labels, which makes cross-dataset comparison difficult [13]. Other surveys also note that many public datasets do not fully represent the behavior captured by medium-interaction honeypots [15]. For this reason, TrapPot uses public datasets for model preparation and the implemented honeypot pipeline for live operational testing.

### **2.1.2 Related Work**

The following studies are closely related to TrapPot because they connect honeypots, IoT attacks, log analysis, and machine learning classification.

Lee and colleagues used Cowrie honeypots in IoT smart factory environments and applied Random Forest classification to distinguish brute-force, scanning, and malware download attacks with 96 percent accuracy and 0.1 milliseconds of inference time [2]. Their work is important to TrapPot because it validates the combination of Cowrie and Random Forest for IoT attack

classification. However, TrapPot extends the project context by targeting a portable academic environment and using five classes, including command execution and normal traffic. Alajrami and Abu-Naser developed an AI-powered intrusion detection system that integrates Cowrie with Random Forest, Logistic Regression, and Isolation Forest for SSH and Telnet attack detection [16]. Their system uses a similar foundation, while TrapPot keeps the learning pipeline simpler by focusing on one trained Random Forest model and a Docker-based deployment.

Other studies validate Random Forest and ensemble learning for IoT intrusion detection without necessarily integrating them with honeypots. Saied and colleagues reported 99.99 percent accuracy for multi-class IoT botnet classification using Random Forest, outperforming several other machine learning methods [17]. Alzahrani and colleagues developed lightweight Random Forest methods with optimized feature engineering and achieved 98 percent accuracy for IoT devices with limited resources [12]. These studies support the algorithm choice in TrapPot, although their datasets and deployment assumptions differ from honeypot-based threat intelligence.

Research on IoT honeypot deployment also supports TrapPot’s design. Saputro and colleagues used Cowrie to build medium-interaction honeypots for IoT environments and reported CPU usage below 6.3 percent, which supports the feasibility of Cowrie-based monitoring in constrained environments [18]. Surnin and colleagues studied honeypot fingerprinting methods that attackers can use to identify decoy systems [19]. These studies show why TrapPot must balance realism, resource use, and isolation while still providing useful data for analysis.

Systematic reviews further support the project direction. Lanz and colleagues reviewed machine learning optimization for IoT honeypots and reported that Random Forest can reach 96 percent accuracy, identifying supervised learning as an effective method for honeypot attack classification [20]. Rodriguez-Gomez and colleagues emphasized that honeypot log management needs automated classification because manual analysis does not scale well [14]. Franco and colleagues surveyed IoT honeypots and honeynets and confirmed that medium-interaction honeypots with Telnet and SSH emulation remain common in the field [10]. Together, these works support TrapPot’s use of Cowrie, Random Forest, and structured log processing.

While existing research validates individual parts of TrapPot, the project combines them into one practical workflow: Cowrie for IoT-like SSH and Telnet deception, Zeek for network-flow evidence, Random Forest classification for five classes, and ELK dashboards for monitoring and presentation. This combination gives the project a clear educational and operational purpose

within the Saudi Arabian academic context.

### **2.1.3 Technical Background**

To support the system design discussed in the Methodology chapter, this section defines the main technologies and concepts used in TrapPot.

#### **2.1.3.1 Honeypots**

A honeypot is a cybersecurity mechanism designed to act as a decoy system that attracts and engages attackers. By mimicking vulnerable services, it captures malicious activity without risking production assets. Honeypots are commonly categorized as low, medium, or high interaction. Medium-interaction honeypots provide a useful balance because they offer believable attacker interaction while keeping the environment controlled [21]. They also provide threat intelligence and labeled data that can support machine-learning-based detection.

#### **2.1.3.2 Cowrie Honeypot**

Cowrie is an open-source medium-interaction SSH and Telnet honeypot that emulates a UNIX-like shell environment. It records authentication attempts, executed commands, downloaded files, and session metadata [22]. Cowrie stores events in structured formats such as JSON, which makes its logs suitable for processing, visualization, and attack classification.

#### **2.1.3.3 SSH and Telnet Protocols**

Secure Shell (SSH) and Telnet are network protocols used for remote command-line access. SSH provides encrypted communication, while Telnet transmits data in plaintext. Both remain important attack vectors in IoT environments because automated bots often try default or weak credentials against exposed services [23]. The Mirai botnet compromised more than 600,000 IoT devices by exploiting weak credentials and exposed remote access services [1].

#### **2.1.3.4 ELK Stack**

The ELK Stack consists of Elasticsearch for search and storage, Logstash for log ingestion and processing, and Kibana for visualization. The stack allows logs from different components to be processed and explored through a central interface [24]. In TrapPot, ELK stores Cowrie, Zeek, and AI detector data and presents the results through dashboards.

### **2.1.3.5 Random Forest Algorithm**

Random Forest is an ensemble learning algorithm that combines multiple decision trees for classification. Each tree learns from a random subset of data and features, and the final class is selected through majority voting. This method reduces overfitting and performs well on structured, high-dimensional datasets. Published IoT intrusion detection studies report Random Forest accuracy in the 96 to 99 percent range while maintaining low inference latency [2, 17].

### **2.1.3.6 Zeek Network Security Monitor**

Zeek is an open-source network analysis framework that works as a passive network monitor. Instead of only matching signatures, Zeek extracts metadata from network traffic, such as connection duration, packet counts, byte counts, service information, and connection state. In TrapPot, Zeek observes traffic interacting with Cowrie and produces `conn.log` and `ssh.log`. These logs provide the network-level features required by the Random Forest detector.

### **2.1.3.7 Supervised Learning**

Supervised learning trains models using labeled datasets where inputs are paired with correct outputs. The model learns patterns during training and then applies those patterns to classify new data. TrapPot uses supervised classification because the Random Forest model learns from labeled network-traffic examples and predicts the class of new Zeek connection records.

### **2.1.3.8 Feature Extraction**

Feature extraction converts raw log data into numerical and categorical values that a machine learning model can use. TrapPot extracts features such as ports, protocol, service, duration, byte counts, packet counts, and connection state from Zeek logs. Well-designed features improve classification because they represent attack behavior in a compact form, while unnecessary or inconsistent features can reduce performance.

### **2.1.3.9 Classification Metrics**

Model evaluation commonly uses accuracy, precision, recall, F1-score, and the confusion matrix. Accuracy measures the overall percentage of correct predictions. Precision measures how many predicted positives are correct, while recall measures how many actual positives the model

detects. The F1-score balances precision and recall. A confusion matrix shows how predicted labels compare with true labels across all classes.

#### **2.1.3.10 Attack Types**

TrapPot classifies five traffic classes. Brute-force attacks involve repeated username and password attempts. Scanning activity involves probing systems to identify active ports, services, and weaknesses. Malware download activity involves attempts to upload or retrieve malicious files. Command execution occurs when an attacker gains shell access and runs commands on the emulated system. Normal traffic represents baseline or benign connection behavior used by the model to separate malicious activity from ordinary network records.

## **2.2 Chronological Evolution and Progress of IoT Threat Detection Systems**

### **2.2.1 Related Work**

IoT security became a distinct research problem in the mid-2010s as researchers documented attacks targeting Telnet and SSH services. Pa and colleagues developed IoTPOT, one of the early honeypots designed for IoT devices, and captured Telnet attacks against ARM, MIPS, and PowerPC architectures. Their work showed that Telnet scanning increased significantly from 2014 onward and confirmed that IoT devices represented a growing attack surface. Cowrie was released in 2015 as an open-source medium-interaction SSH and Telnet honeypot [22]. Cowrie is directly relevant to TrapPot because it provides the technical foundation for the honeypot layer.

The IoT security landscape changed sharply in 2016 with the Mirai botnet, which compromised more than 600,000 IoT devices by exploiting weak default credentials on exposed remote access services. Antonakakis and colleagues documented how Mirai scanned for exposed services and used only 62 hardcoded credential pairs to gain access [1]. This study is important to TrapPot because it explains why Telnet and SSH remain central to IoT attack monitoring. Famera and colleagues later discussed Mirai variants such as Satori and Moobot, showing that the same style of weakness continues to matter [3].

Researchers then focused on integrating honeypots with automated classification. Saputro and colleagues deployed medium-interaction honeypots using Cowrie and showed that CPU load

remained below 6.3 percent [18]. Lee and colleagues applied Random Forest classification to IoT smart factory honeypot data and achieved 96 percent accuracy for brute-force, scanning, and malware download attacks with 0.1 milliseconds of inference latency [2]. Franco and colleagues surveyed IoT honeypots and found that medium-interaction systems prioritizing Telnet and SSH are common in the field [25].

Recent research from 2023 to 2025 further supports ensemble learning approaches. Saied and colleagues reported 99.99 percent accuracy for multi-class IoT botnet classification using Random Forest [17]. Lanz and colleagues reviewed machine learning optimization for IoT honeypots and confirmed that Random Forest can reach 96 percent accuracy in this domain [20]. Rodriguez-Gomez and colleagues emphasized the need for automated classification to manage large volumes of honeypot logs [14]. Alajrami and Abu-Naser also developed an AI-powered intrusion detection system that integrates Cowrie with Random Forest for SSH and Telnet attack detection [16].

TrapPot builds on this progression by adopting Cowrie’s medium-interaction architecture, focusing on Telnet and SSH protocols identified by Mirai-related research, using Zeek to produce structured traffic evidence, and applying Random Forest classification supported by recent IoT detection studies. The result is a practical proof of concept for automated IoT attack monitoring that is feasible for undergraduate implementation and aligned with local cybersecurity capacity-building goals.



## **Chapter 3**

# **METHODOLOGY**

# METHODOLOGY

This chapter describes the approach used to design, implement, and evaluate the TrapPot system [10, 14, 20]. The methodology covers the research design, data collection procedures, pre-processing and feature engineering pipeline, Random Forest classifier training, validation plan, and ethical considerations [2, 11, 26]. The chapter explains how TrapPot captures IoT-targeted activity, converts it into useful features, and evaluates classification performance against the project target of at least 99% accuracy [2, 26, 27].

## 3.1 Research Design

The TrapPot study follows a case study research design with experimental elements, supported by a quantitative approach [10, 14]. This design fits the project because TrapPot is a built system rather than a survey-only study. The research examines how a specific implementation captures attacker behavior, processes logs, and classifies network traffic under controlled conditions [2, 20]. The quantitative approach is used because system logs, feature vectors, and model metrics can be measured and compared statistically [11, 28].

### 3.1.1 Research Design Type: Case Study

The primary research design is a case study of one concrete deployment: the TrapPot system running in a dedicated and isolated environment [29, 9]. This design allows detailed observation of how medium-interaction honeypot technology and machine learning behave when combined in one practical IoT security workflow [2, 18].

By focusing on one deployment, the case study makes it possible to observe [14, 20]:

- how attackers interact with emulated Telnet and SSH services [22, 23, 3],
- how logs move through the ELK-based data pipeline [24, 30], and
- how the Random Forest classifier responds to brute-force, scanning, malware download, command execution, and normal traffic patterns [2, 26, 17].

This depth of analysis is more suitable for a system-development project than a broad theoretical survey because it evaluates the practical integration of honeypots, network monitoring, and AI-based classification.

### 3.1.2 Research Approach: Quantitative

The research uses quantitative methods because the collected evidence is machine-generated and structured. Cowrie records authentication attempts, commands, sessions, and file activity, while Zeek records ports, protocols, services, packet counts, byte counts, connection duration, and connection state [22, 30]. These records are converted into feature vectors that can be used for statistical analysis and model evaluation [31, 32].

The methodology follows a system-development and experimental-validation pattern [2, 14]:

- Develop and deploy TrapPot using Cowrie, Zeek, the ELK Stack, and a Random Forest classifier in an isolated network [18, 9].
- Collect attack traffic data over the specified operational period [2, 6].
- Train and test the Random Forest model using the prepared dataset [17, 26].
- Evaluate the model using accuracy, precision, recall, F1-score, and confusion matrix results [33, 28].

This approach allows the team to measure both the classification performance of the model and the operational behavior of the complete TrapPot pipeline.

## 3.2 Data Collection

TrapPot collects and uses data in two phases. The first phase uses existing research and public datasets to prepare and validate the AI module. The second phase uses logs generated by the deployed honeypot and monitoring system for live classification and operational evaluation [14, 20].

### 3.2.1 Pre-Implementation Data Collection

Before live deployment, secondary data from public and academic sources was used to design, train, and validate the AI module [11, 13, 15]. This makes the model development stage grounded in real IoT security patterns and established research.

#### **Sources of Secondary Data:**

##### **Academic Publications & Research Datasets**

- Lee et al. (2021) provided labeled attack logs from Cowrie honeypots in IoT smart factory environments, classifying brute-force, scanning, and malware download attacks with 96% accuracy using Random Forest [2].
- Saputro et al. (2021) supplied performance benchmarks and attack traces from medium-interaction IoT honeypots, confirming Cowrie’s low resource overhead below 6.3% CPU [18].
- Saied et al. (2023) offered comparative evidence for IoT botnet classification, demonstrating Random Forest’s 99.99% accuracy for multi-class attack detection [17].
- Alajrami & Abu-Naser (2025) shared an SSH/Telnet honeypot-based intrusion detection approach that supports testing multi-algorithm IDS performance [16].

### **Public IoT Security Datasets**

- Mirai botnet traces provide historical attack data from Mirai and variants such as Gafgyt and Mozi, focusing on credential-based attacks against Telnet and SSH [1, 3].
- Cowrie public logs provide examples of SSH and Telnet honeypot sessions that can support feature extraction and model preparation [22, 23].
- IoT-23 and CIC-IoT-2023 provide labeled IoT network traffic datasets that help validate feature relevance and model generalization [34, 13, 15, 35].

### **Technical Documentation & Cybersecurity Reports**

- Cowrie documentation provides JSON log structures, command emulation behavior, and session metadata formats [22, 30].
- ELK Stack guidance supports Logstash pipelines and Elasticsearch mappings for parsing and storing honeypot data [24].
- Threat intelligence reports and academic surveys provide context on scanning patterns, malware delivery, and IoT attack trends [5, 7].

#### **3.2.2 Post-Implementation Data Collection**

After deployment in an isolated network environment, TrapPot collects primary operational data in real time [2, 6]. This data is used to:

- feed the live classification module,
- support future model refinement and validation, and
- provide current threat intelligence for analysis and reporting.

The implementation and testing details of this phase are presented in Chapter 4, while the observed results are discussed in Chapter 5.

## 3.3 Data Analysis

### 3.3.1 Data Acquisition and Preprocessing Pipeline

The data acquisition pipeline relies on Zeek transport-layer outputs. Cowrie acts as the deception layer, while Zeek monitors network connections and generates the tabular stream used by the AI detector [22, 30]. The ELK Stack, consisting of Elasticsearch, Logstash, and Kibana, handles processing, storage, and visualization [24]. Logstash processes raw JSON records and extracts fields such as session identifiers, timestamps, source IP addresses, commands, and credential-attempt information [24, 30].

**Data Cleaning and Organization:** To maintain data integrity, records are organized by session ID and event timestamp. This helps identify repeated records and preserve the relationship between attacker actions and network events [14]. Missing command or input fields are handled during feature calculation so incomplete sessions do not produce invalid model input [32, 28].

**Ground Truth Labeling Strategy:** As a supervised learning project, accurate labeling is essential [36, 31]. A hierarchical strategy assigns one of five target classes to each session or connection record: Brute Force, Scanning, Malware Download, Command Execution, or Normal traffic [2, 26].

- Malware Download is identified by the presence of data in the URLs field [22].
- Command Execution requires evidence of a successful login followed by executed commands [22, 23].
- Brute Force is labeled when there is a high frequency of failed login attempts [1, 3].
- Scanning is categorized by low-interaction sessions that show port or service probing behavior [4, 6].

- Normal traffic represents baseline operational events, minor polling, or protocol checks with minimal packet counts and predictable transport signatures [2].

### 3.3.2 Feature Engineering and Selection

Feature engineering transforms categorical and sequential log data into a compact set of numerical and categorical features that can separate the five classes [31, 35]. These features must be meaningful to the Random Forest model and understandable to the project team, so the prototype avoids complex dimensionality reduction techniques [2, 31].

The primary features are based on authentication behavior, session activity, command usage, file transfer indicators, and network connection properties [35, 26]:

**Authentication Metrics** include the number of unique login attempts per session. A high value indicates automated brute-force activity that tests many credential pairs [1, 3, 23].

**Session Metrics** include session duration in seconds. Short durations can indicate automated scanning, while longer sessions may indicate interactive command execution [2, 26].

**Command Metrics** focus on post-login behavior, especially the ratio of recognized operating system commands to invalid or empty inputs. This provides evidence of command execution after access is obtained [22, 31].

**Malware Metrics** use a binary file-transfer flag to indicate whether an upload or download attempt occurred. This feature supports identification of malware download behavior [22, 23].

**Network Metrics** are derived from Zeek `conn.log`, including source and destination ports, duration, byte counts, packet counts, protocol, service, and connection state. These features match the trained detector pipeline and allow live classification from Zeek records.

This feature set supports the project goal of interpretable and efficient classification while remaining compatible with the implemented detector [2, 17, 26].

### 3.3.3 Machine Learning Model Training and Optimization

The analysis relies on the Random Forest Classifier, a choice supported by literature reporting high accuracy in IoT intrusion detection and low inference latency [2, 17, 26, 35]. The prepared dataset is partitioned into an 80% training set and a 20% hold-out testing set [37, 26]. Five-fold cross-validation is used during training to evaluate model robustness across the training data [38, 37]. To reduce the effect of class imbalance, the model uses class weighting and balancing strategies so that less frequent classes such as Malware Download are not ignored [39, 40, 41].

Hyperparameters, including the number of decision trees and maximum tree depth, are tuned to support the project's minimum 99% classification accuracy goal [38, 37].

### 3.4 Validation Plan

The validation plan measures both the analytical accuracy of the Random Forest classifier and the operational feasibility of TrapPot as a real-time threat intelligence system [33, 28]. The main success target is classification accuracy above 99% [2, 26, 27].

#### 3.4.1 Classification Performance Metrics

Accuracy is the primary metric because it directly measures the percentage of correct predictions across the five classes [26, 28]. However, security datasets often contain imbalanced classes, so additional metrics are also needed [33, 28].

**Primary Metric:** Accuracy is used to determine whether the model achieves the 99% classification target across the five attack categories [26, 28].

**Secondary Critical Metric:** F1-score is reported alongside accuracy because it balances precision and recall [33, 26]. For TrapPot, a high F1-score helps confirm that less frequent but important classes, such as Malware Download and Command Execution, are not hidden by more frequent brute-force or normal traffic records [2, 39].

#### 3.4.2 Validation Dataset and Testing Method

Final model validation uses the 20% hold-out dataset that was separated during data preparation [37, 26]. This data is not used during training, which gives a fairer measure of model generalization [2, 17, 27].

**Data Collection Duration:** The study design keeps the 30-day operational data collection period for the Cowrie honeypot in an isolated network environment [2, 6]. This duration is intended to provide a stronger sample of attack traffic targeting Telnet and SSH [3, 23].

**Methodology:** The trained Random Forest model is applied to the hold-out set, and the resulting predictions are compared against the ground truth labels established during preprocessing [33, 28].

### 3.4.3 Operational Success Criteria

Beyond analytical metrics, TrapPot must demonstrate that it can operate as a real-time monitoring and classification system [14, 20]. Two system-level criteria are used:

**Real-Time Inference Latency:** The trained Random Forest model must classify new connection records with low latency so that Kibana can support near-real-time monitoring through the ELK Stack [24, 26].

**Resource Efficiency:** Cowrie, Zeek, Logstash, Elasticsearch, Kibana, and the Random Forest detector must run with acceptable CPU and memory use for a local academic deployment [18, 12]. This validates the system’s suitability for educational and non-enterprise lab environments [10, 8].

### 3.4.4 Results Presentation

The system results are presented through both quantitative tables and visual monitoring outputs [33, 28]. A confusion matrix is used to show how predicted labels compare with actual labels across the five classes [33, 26, 28]. Kibana dashboards are used to show operational evidence, including attack activity over time, source IP addresses, protocol distribution, commands captured, and AI detector output [24].

## 3.5 Research Ethics

TrapPot follows a do-no-harm ethical approach. The project observes malicious behavior for research and learning, but it must not endanger university systems, external networks, or identifiable individuals [9, 10]. Because the project does not involve human-subject experiments, the main ethical concerns are network containment, responsible log handling, and careful reporting [10, 42].

### 3.5.1 Network Isolation and Containment

Network safety is the main ethical and technical requirement [9, 29]. Cowrie is designed to capture attack data, but the deployment must not become a risk to the university network or a launchpad for third-party attacks [9, 43].

**Strict Segregation:** TrapPot is deployed inside a dedicated and isolated lab network or virtual network segment, separated from production infrastructure [29, 43].



**Preventing Egress:** Firewall and network controls must restrict outgoing traffic from the honeypot environment so that attackers cannot use the simulated system to attack external targets [9].

**Sandbox Security:** Cowrie’s medium-interaction design provides a controlled shell environment. Attacker commands are recorded inside the honeypot and do not execute on the host operating system [22, 18].

### 3.5.2 Data Handling and Anonymization

Captured logs may include source IP addresses, timestamps, credentials attempted by attackers, and commands entered during sessions [14, 2]. The project therefore keeps only the data needed for threat intelligence, classification, and academic reporting.

**Data Minimization:** TrapPot stores the fields required for analysis, such as source IP address, timestamp, credential attempt, command input, and session metadata [14, 2].

**Storage and Access:** Log data stored in ELK and project machines is restricted to the project team and used only for research and evaluation [24].

**Anonymity in Reporting:** Public reporting uses aggregated statistics and anonymized summaries. Individual raw source IP addresses are not disclosed in the thesis, presentation, or public materials [10, 14].

### 3.5.3 Regulatory Compliance and Log Management

**Institutional Approval:** Any 30-day data collection period or external exposure must be approved by the relevant university or lab authority to confirm that the deployment and isolation plan comply with security policies [10, 42].

**Data Disposal:** After project completion, raw identifiable logs should be purged from ELK and associated storage. Only anonymized and aggregated summaries should be retained for future comparison or academic reference [14, 10].

## Chapter 4

# IMPLEMENTATION AND TESTING

## IMPLEMENTATION AND TESTING

This chapter presents the implementation and testing of the TrapPot system. The implementation turns the design from the previous chapters into a Docker-based IoT honeypot environment that captures SSH and Telnet interaction, monitors the related network traffic, classifies traffic with a trained Random Forest model, and visualizes the collected evidence in Kibana [2, 22, 23]. The chapter describes the development environment, system architecture, component configuration, feature extraction algorithms, deployment workflow, and verification tests.

All implementation code, configuration files, and deployment files are version-controlled in the project repository at: <https://github.com/bxrayxd/graduation-project>.

### 4.1 Development Environment and Tools

The implementation uses established open-source tools so that the system remains reproducible and understandable for an academic project [11, 14, 20].

#### 4.1.1 Primary Development Platform

**Python 3.10+** is used for the custom parts of TrapPot, including feature extraction, model training, and live classification [37, 26]. Python is suitable for this work because it provides mature machine learning and data-processing libraries such as scikit-learn, pandas, NumPy, and joblib [31, 2].

**scikit-learn 1.3+** provides the Random Forest implementation and evaluation utilities, including train-test splitting, cross-validation, and performance metrics [2, 17, 26]. **pandas** and **NumPy** are used to prepare structured traffic data, handle missing values, and arrange features in the correct order for the model.

#### 4.1.2 Containerization and Orchestration

**Docker** and **Docker Compose** are used to run the complete TrapPot stack as a set of containers [18, 9]. The current Docker Compose deployment includes Cowrie, Zeek, the AI detector, Elasticsearch, Logstash, Kibana, `elastic_setup`, and `kibana_setup`. This makes the system easier to reproduce across machines and keeps dependencies separated from the host operating system.

Docker Compose manages service networking, shared volumes, service startup order, and persistent Elasticsearch storage. It also lets Zeek share the Cowrie network namespace so Zeek can monitor SSH and Telnet traffic directed at the honeypot.

### 4.1.3 Data Processing and Visualization Tools

**Cowrie** provides the medium-interaction SSH and Telnet honeypot layer [22, 18, 23]. It records authentication attempts, command input, sessions, and download attempts in structured logs.

**Zeek** provides the network monitoring layer. It produces `conn.log` and `ssh.log`, which summarize connection-level and SSH-level activity. These logs provide the twelve network features used by the live detector.

**Elasticsearch, Logstash, and Kibana** provide the storage and visualization layer [24, 30]. The current implementation uses Elastic Docker images version 9.4.0. Elasticsearch stores TrapPot indices, Logstash parses and forwards Cowrie, Zeek, and detector logs, and Kibana provides data views and the **TrapPot Overview** dashboard.

## 4.2 System Architecture

TrapPot follows a modular architecture built around four functions: capture, monitor, classify, and visualize. Cowrie captures attacker interaction, Zeek monitors network traffic, the AI detector classifies Zeek connection records, and the ELK Stack stores and displays the results [10, 14, 20].

### 4.2.1 High-Level Architecture Overview

Figure 4.1 shows the main data flow from honeypot interaction to dashboard visualization.

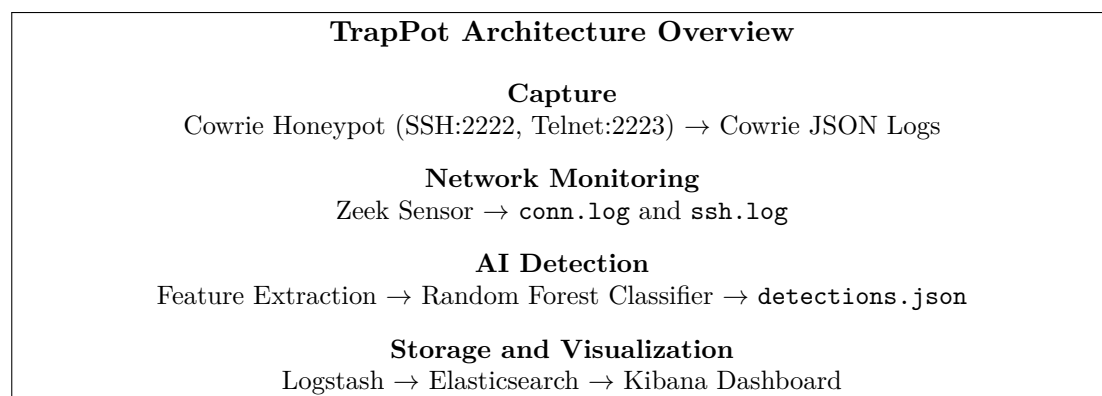


Figure 4.1: TrapPot system architecture and data flow.

**Capture Layer:** Cowrie listens on TCP 2222 for SSH and TCP 2223 for Telnet. It records login attempts, shell commands, session activity, and file download attempts in JSON logs [22, 30].

**Network Monitoring Layer:** Zeek monitors traffic in Cowrie's network namespace and writes structured connection records to `conn.log` and SSH metadata to `ssh.log`. This layer converts raw network traffic into the structured input required by the AI detector.

**Intelligence Layer:** The AI detector monitors Zeek's `conn.log`, extracts the twelve expected features, loads `trappot_model.pkl` and `label_encoder.pkl`, and writes predictions to `detections.json`.

**Storage and Visualization Layer:** Logstash reads Cowrie logs, Zeek logs, and detector output, then sends them to Elasticsearch indices. Kibana uses prepared data views and dashboards to display the collected activity.

#### 4.2.2 Component Interaction and Data Flow

The system uses a real-time data flow. When a connection reaches the honeypot, Cowrie records the interaction and Zeek records the network connection. The AI detector reads new Zeek connection rows, prepares the values for the trained Random Forest model, prints the prediction as a live alert, and saves the result. Logstash then ships the available logs to Elasticsearch, where Kibana can display them through the dashboard.

#### 4.2.3 Network Topology and Isolation

The TrapPot solution runs locally through Docker Compose. Cowrie, the AI detector, Logstash, Elasticsearch, Kibana, and the setup containers communicate through a Docker bridge network. Zeek uses `network_mode: "service:cowrie"` so it can observe Cowrie traffic directly.

Only the required local testing ports are exposed: Cowrie SSH on TCP 2222, Cowrie Telnet on TCP 2223, and Kibana on TCP 5601. This supports controlled local testing and dashboard access without requiring public deployment. The system should be used only in an approved lab network and should not be connected to public or university infrastructure without authorization.

## 4.3 Component Implementation

### 4.3.1 Cowrie Honeypot Deployment

Cowrie is deployed using the official `cowrie/cowrie` Docker image with project-specific configuration files mounted into the container [22, 18]. The main configuration file is `cowrie.cfg`. Login rules are stored in `userdb.txt`.

**Configuration highlights include:**

- SSH and Telnet exposure on host ports 2222 and 2223.
- A realistic Linux identity through customized hostname, OS release, issue, MOTD, and selected fake filesystem files.
- JSON session logging for authentication attempts, commands, and file download activity.
- Persistent Cowrie log storage through a Docker volume shared with Logstash.

### 4.3.2 Zeek Sensor Deployment

Zeek is deployed using the `blacktop/zeek` Docker image. It is attached to Cowrie's network namespace and runs with checksum validation disabled through `-C`, which is appropriate for containerized packet capture where checksum offloading can otherwise interfere with analysis. The command also sets the local Docker address range so Zeek can classify the lab network correctly.

Zeek writes output logs into the shared `zeek/logs` directory. The two most important files for TrapPot are `conn.log`, which contains connection records, and `ssh.log`, which contains SSH protocol metadata.

### 4.3.3 Machine Learning Module Implementation

The machine learning module consists of the detector script, the trained model file, and the label encoder file. The detector runs in a dedicated container and watches Zeek's `conn.log`. For each new connection row, it extracts twelve features:

- `id.orig_p`: originator port,
- `id.resp_p`: responder port,

- **duration:** total connection duration,
- **orig\_bytes:** bytes sent by the originator,
- **resp\_bytes:** bytes sent by the responder,
- **orig\_pkts:** packets sent by the originator,
- **orig\_ip\_bytes:** IP bytes sent by the originator,
- **resp\_pkts:** packets sent by the responder,
- **resp\_ip\_bytes:** IP bytes sent by the responder,
- **proto:** transport protocol,
- **service:** detected service, and
- **conn\_state:** Zeek connection state.

The model was trained using the IoT-23 dataset, which contains labeled malicious and benign IoT network traffic [34]. The training process selected Zeek-compatible features, handled missing values, encoded categorical fields, used synthetic brute-force samples and SMOTE to reduce imbalance, trained the Random Forest classifier, and saved the trained model and encoder.

#### 4.3.4 ELK Stack Configuration

Elasticsearch is configured as a single-node secured cluster. Security is enabled, HTTP SSL is disabled for local lab simplicity, and passwords are provided through the `TrapPot/.env` file. The `elastic_setup` service creates the Kibana system password, a limited `trappot_writer` user for Logstash, and index templates for the TrapPot indices.

Logstash reads four inputs: Cowrie JSON logs, Zeek `conn.log`, Zeek `ssh.log`, and detector `detections.json`. It parses and normalizes fields, converts numeric values where needed, applies GeoIP enrichment when a public source IP is available, and writes documents to these Elasticsearch indices:

- `trappot-cowrie`
- `trappot-zeek-conn`
- `trappot-zeek-ssh`

- `trappot-detector`

Kibana is configured with prepared data views and the `TrapPot Overview` dashboard. The `kibana_setup` service imports these saved objects through Kibana’s documented HTTP API after Kibana starts.

## 4.4 Feature Extraction and Classification Algorithms

This section presents the main algorithms used by TrapPot for feature extraction, model training, and live classification.

### 4.4.1 Feature Extraction Algorithm

Algorithm 1 shows how a Zeek `conn.log` entry is transformed into the twelve-feature record required by the Random Forest classifier.

---

#### Algorithm 1 Zeek Connection Feature Extraction

---

**Require:** New Zeek connection log entry  $L$  from `conn.log`  
**Ensure:** Ordered feature record  $F$  containing twelve features

- 1: Read new line  $L$  from `conn.log`
- 2: **if**  $L$  starts with “#” **then**
- 3:   Skip the line
- 4: **end if**
- 5:  $parts \leftarrow \text{split}(L, \text{tab})$
- 6: **if**  $\text{len}(parts) \geq 20$  **then**
- 7:    $F[id.orig\_p] \leftarrow parts[3]$
- 8:    $F[id.resp\_p] \leftarrow parts[5]$
- 9:    $F[duration] \leftarrow \text{clean\_number}(parts[8])$
- 10:    $F[orig\_bytes] \leftarrow \text{clean\_number}(parts[9])$
- 11:    $F[resp\_bytes] \leftarrow \text{clean\_number}(parts[10])$
- 12:    $F[orig\_pkts] \leftarrow \text{clean\_number}(parts[16])$
- 13:    $F[orig\_ip\_bytes] \leftarrow \text{clean\_number}(parts[17])$
- 14:    $F[resp\_pkts] \leftarrow \text{clean\_number}(parts[18])$
- 15:    $F[resp\_ip\_bytes] \leftarrow \text{clean\_number}(parts[19])$
- 16:    $F[proto] \leftarrow parts[6]$
- 17:    $F[service] \leftarrow parts[7]$
- 18:    $F[conn\_state] \leftarrow parts[11]$
- 19: **end if**
- 20: Arrange  $F$  according to the model feature order
- 21: **return**  $F$

---

The function `clean_number` converts Zeek missing-value symbols such as `-` into numeric zero so the detector does not pass invalid values to the model.



### 4.4.2 Random Forest Training Algorithm

Algorithm 2 summarizes the training pipeline used for the Random Forest model [2, 17, 37].

---

**Algorithm 2** Random Forest Classifier Training
 

---

**Require:** Labeled dataset  $D = \{(X_i, y_i)\}_{i=1}^N$

**Ensure:** Trained Random Forest model  $M$

```

1:  $D_{train}, D_{test} \leftarrow \text{split}(D, \text{ratio}=0.8)$ 
2: Encode categorical fields and clean missing values
3: Balance selected classes using synthetic samples and SMOTE
4:  $grid \leftarrow \{n\_estimators, max\_depth\}$ 
5:  $best\_params \leftarrow \text{GridSearchCV}(\text{RandomForest}, grid, cv=5)$ 
6:  $M \leftarrow \text{RandomForestClassifier}(best\_params, class\_weight='balanced')$ 
7:  $M.\text{fit}(D_{train})$ 
8:  $predictions \leftarrow M.\text{predict}(D_{test})$ 
9:  $accuracy \leftarrow \text{accuracy\_score}(D_{test}.y, predictions)$ 
10:  $f1 \leftarrow \text{f1\_score}(D_{test}.y, predictions, average='weighted')$ 
11:  $cm \leftarrow \text{confusion\_matrix}(D_{test}.y, predictions)$ 
12: return  $M$ 

```

---

The 80/20 split gives a hold-out test set, while five-fold grid search helps tune the model without manually selecting parameters [37, 26]. Balanced class weighting reduces bias toward majority classes [39, 41, 44].

### 4.4.3 Real-Time Classification Algorithm

Algorithm 3 describes the live detector loop.

---

**Algorithm 3** Real-Time Attack Classification Loop

---

**Require:** Trained model  $M$ , label encoder  $LE$ , Zeek log file `conn.log`**Ensure:** Live classification alerts and detection records

```

1:  $M \leftarrow \text{load\_model}(\text{trappot\_model.pkl})$ 
2:  $LE \leftarrow \text{load\_encoder}(\text{label\_encoder.pkl})$ 
3: Wait until conn.log exists
4: Open conn.log in monitoring mode
5: while True do
6:   Read next new line from conn.log
7:   if no new line exists then
8:     Wait briefly and continue
9:   end if
10:  if line starts with “#” then
11:    Skip the line
12:  end if
13:   $F \leftarrow \text{extract\_features}(\text{line})$ 
14:  Preprocess numerical and categorical values
15:   $\text{prediction} \leftarrow M.\text{predict}(F)$ 
16:  Print live alert containing the predicted class
17:  Write detection result to detections.json
18: end while

```

---

The detector reads only newly appended log entries instead of reprocessing the whole file. This supports live monitoring and avoids unnecessary repeated work.

## 4.5 System Integration and Deployment

All TrapPot services are orchestrated through the Docker Compose file in the `TrapPot` directory. The deployment uses shared volumes for Cowrie logs, Zeek logs, detector results, and Elasticsearch data. Service dependencies ensure that Elasticsearch becomes healthy before setup services create users, roles, and templates, and that Kibana starts before the dashboard objects are imported.

The complete environment is started with:

```
docker compose up --build -d
```

After startup, users can connect to Cowrie through SSH on port 2222 or Telnet on port 2223, then open Kibana through `http://localhost:5601`. The configured Elastic credentials are read from `TrapPot/.env`.

### 4.5.1 Deployment Verification

Post-deployment verification checks the following:

- **Container Verification:** `docker compose ps` confirms that Cowrie, Zeek, the AI detector, Elasticsearch, Logstash, Kibana, and setup services started successfully.
- **Cowrie Health Check:** An SSH session to `localhost:2222` confirms that Cowrie accepts the test connection and records commands.
- **Zeek Health Check:** The files `conn.log` and `ssh.log` appear in the Zeek log directory after the SSH test.
- **AI Detector Health Check:** The detector reads `conn.log` and prints live Random Forest classification output.
- **ELK Stack Health Check:** Elasticsearch contains TrapPot indices, and Kibana provides the TrapPot data views and dashboard.

## 4.6 Testing Strategy

TrapPot was tested through component-level checks and integration checks. The purpose was to verify that interaction captured by Cowrie becomes Zeek traffic evidence, AI detector output, Elasticsearch records, and Kibana dashboard data [33, 28, 11].

### 4.6.1 Unit Testing

**Cowrie Testing:** Cowrie was tested by connecting through SSH on port 2222 and running commands such as `whoami`, `pwd`, and `ls -la`. These commands confirmed that Cowrie emulated a Linux shell and recorded attacker-like behavior.

**Zeek Testing:** Zeek was tested by checking that `conn.log` and `ssh.log` were created after the Cowrie SSH session.

**AI Detector Testing:** The AI detector container logs were reviewed after Zeek generated connection entries. The detector read `conn.log`, generated features, and printed real-time classification alerts.

**ELK Testing:** Logstash and Elasticsearch were tested by checking that TrapPot indices were created. Kibana was then opened to verify that data views and the dashboard were available.

### 4.6.2 Integration Testing

**Cowrie-to-Zeek Integration:** An SSH session was initiated against Cowrie on port 2222. Zeek observed the generated traffic and wrote connection records to `conn.log` and SSH meta-data to `ssh.log`.

**Zeek-to-AI Integration:** The AI detector consumed new entries from `conn.log`, confirming that it could classify live traffic produced during honeypot interaction.

**Logstash-to-Elasticsearch Integration:** Logstash processed Cowrie, Zeek, and detector logs and forwarded them to the expected Elasticsearch indices.

**Elasticsearch-to-Kibana Integration:** Kibana connected to Elasticsearch and displayed the TrapPot data views and dashboard through the browser interface.

### 4.6.3 Performance Testing

The performance test focused on whether the stack could run smoothly in the local lab environment. Docker Compose started the services, Zeek created connection logs after SSH activity, the detector classified new records, and Kibana remained available through `localhost:5601`. Because Elasticsearch, Logstash, and Kibana require more memory than Cowrie and the AI detector, the host machine must allocate enough memory for the full stack.

### 4.6.4 Security Testing

Security testing focused on controlled local use. Exposed services were restricted to SSH on 2222, Telnet on 2223, and Kibana on 5601. Attacker behavior was simulated manually using local SSH sessions instead of exposing the system to the public internet. The documentation states that TrapPot should only be run in a lab network and should not be connected to public or university infrastructure without approval.

### 4.6.5 Classifier Accuracy Validation

The trained Random Forest model was evaluated on the prepared IoT-23 dataset, which contains labeled IoT network traffic flows for intrusion detection research [34]. The training process used selected Zeek-compatible network features, synthetic brute-force samples, and SMOTE balancing to reduce class imbalance. The final trained model achieved approximately 99.59% accuracy for the five selected classes. The model was saved as `trappot_model.pkl` and deployed

in the AI detector container with `label_encoder.pkl`. Future validation should include larger volumes of live honeypot traffic and additional attack scenarios.

## Chapter 5

# RESULTS AND DISCUSSIONS

## RESULTS AND DISCUSSIONS

This chapter presents the results and discussion of the TrapPot implementation and testing. The main objective is to evaluate whether the developed system met its goal of capturing SSH and Telnet interactions, monitoring related network traffic, classifying traffic behavior with the trained Random Forest model, and visualizing the collected data through Kibana.

The analysis treats TrapPot as a complete pipeline rather than a set of separate tools. Cowrie provides honeypot interaction, Zeek provides network monitoring, the Random Forest detector provides traffic classification, and the ELK Stack provides log storage and visualization. The results are therefore discussed according to how well these components worked together during testing.

The chapter is organized into four sections: analysis strategy, analysis results, major findings, and discussion.

### 5.1 Analysis Strategy

TrapPot was evaluated from both operational and machine learning perspectives. Since the project is a cybersecurity monitoring and detection system, success is not measured only by model accuracy. The full path from honeypot interaction to dashboard visualization must also work correctly.

The analysis was guided by the following questions:

- Can Cowrie act as an SSH and Telnet honeypot and log attacker-like interaction?
- Can Zeek monitor the traffic generated by the honeypot and produce structured network logs?
- Can the AI detector read Zeek's `conn.log` and classify traffic behavior using the trained Random Forest model?
- Can Logstash process the generated logs and send them to Elasticsearch?
- Can Kibana display the indexed data through data views and the TrapPot dashboard?

The system was tested in a controlled local lab environment using Docker Compose. After the services started, an SSH session was opened to the Cowrie honeypot on port 2222. During

the test session, commands such as `whoami`, `pwd`, `ls -la`, `cat /etc/passwd`, and `wget` were executed.

These commands were selected because they represent common actions an attacker may take after gaining shell access. For example, `whoami` checks the active user, `pwd` checks the current directory, `ls -la` lists files, `cat /etc/passwd` attempts to read system account information, and `wget` simulates a file download attempt.

After the session, each component was checked. Cowrie logs were reviewed to confirm that the login and commands were recorded. Zeek logs were checked to confirm that `conn.log` and `ssh.log` were generated. The AI detector output was reviewed to confirm that classification alerts appeared. Elasticsearch indices were checked to confirm that logs were indexed, and Kibana was opened to confirm that the data views and dashboard were available.

## 5.2 Analysis Result

The test results showed that the main parts of TrapPot worked as a connected Docker-based system. Docker Compose started the required services, including Cowrie, Zeek, the AI detector, Elasticsearch, Logstash, Kibana, and the setup services. This confirmed that the project can be deployed as a portable multi-container environment.

Cowrie successfully accepted an SSH connection on port 2222. After login, the honeypot provided a fake Linux shell environment and accepted the commands entered during the session. Cowrie recorded the login attempt, successful authentication, executed commands, and file download attempt. This result shows that the honeypot layer achieved its purpose of attracting and capturing attacker-like behavior without exposing a real production system.

Zeek monitored the traffic generated by the Cowrie session. The connection records in `conn.log` showed SSH traffic between the host machine and the Cowrie container. Zeek also captured traffic generated by the simulated `wget` command. This confirms that the network monitoring layer converted traffic activity into structured log records.

The AI detector read Zeek's `conn.log` and produced live classification alerts. During testing, the detector produced output such as `Normal detected`. This validates that the detector could extract the relevant network features, load the trained Random Forest model, and produce a runtime prediction. The detector also wrote detection records to `detections.json`, allowing the detection output to be stored and later indexed.



Logstash and Elasticsearch were also validated. Logstash connected to Elasticsearch and parsed existing TrapPot logs. Elasticsearch contained the expected TrapPot indices for Cowrie, Zeek, and detector output. This confirms that the log processing and indexing layer worked correctly.

Kibana was available at `localhost:5601`. Data views were available for Cowrie logs, Zeek connection logs, Zeek SSH logs, and AI detector results. The **TrapPot Overview** dashboard was also available. This validated that the visualization layer was configured correctly and could be used to inspect collected logs through a browser interface.

Table 5.1 summarizes the main component verification results.

Table 5.1: TrapPot Component Verification Results

| Component     | Verification Method                                                                       | Result |
|---------------|-------------------------------------------------------------------------------------------|--------|
| Cowrie        | SSH login was performed on port 2222 and commands were executed in the fake shell.        | Passed |
| Zeek          | <code>conn.log</code> and <code>ssh.log</code> were generated after the SSH test session. | Passed |
| AI Detector   | The detector read Zeek's <code>conn.log</code> and produced live classification alerts.   | Passed |
| Logstash      | Logstash processed TrapPot logs and connected to Elasticsearch.                           | Passed |
| Elasticsearch | TrapPot-related Cowrie, Zeek, and detector indices were created.                          | Passed |
| Kibana        | Data views and the <b>TrapPot Overview</b> dashboard were available in the browser.       | Passed |

The Random Forest model was tested on the prepared IoT-23 dataset [34]. The final trained model achieved approximately 99.59% accuracy across the five selected classes. This result indicates that the model learned the prepared dataset effectively, although live deployment performance should be validated further with larger volumes of real honeypot traffic.

Table 5.2 presents the model accuracy result.

Table 5.2: Random Forest Model Accuracy Result

| Model                    | Accuracy |
|--------------------------|----------|
| Random Forest Classifier | 99.59%   |

### 5.3 Major Findings

The first major finding is that TrapPot integrated honeypot interaction, network monitoring, machine learning detection, and dashboard visualization into one working environment. The

system did not only collect terminal logs. It also transformed network activity into structured logs and used those logs for live classification.

The second finding is that Zeek is important in linking the honeypot to the AI detector. Cowrie records attacker behavior at the interaction level, such as login attempts and commands. Zeek records network activity, including ports, protocols, services, duration, byte counts, packet counts, and connection state. This makes the detector more useful because it classifies traffic using structured network features rather than raw command logs alone.

The third finding is that the AI detector was successfully connected to the live system. The detector loaded `trappot_model.pkl` and `label_encoder.pkl`, monitored Zeek's `conn.log`, extracted the twelve required features, and produced live classification alerts. This confirms that the machine learning component was integrated with the implemented pipeline and was not only an offline model.

The fourth finding is that Docker Compose improved deployment and portability. A single Docker Compose workflow can start the full system instead of requiring each component to be installed and configured manually.

The fifth finding is that Kibana made the system easier to inspect and present. Terminal logs are useful for debugging, but Kibana provides a clearer interface for searching, filtering, and viewing activity through dashboards.

## 5.4 Discussion

The results show that TrapPot achieved its main implementation goals. The system recorded SSH honeypot activity, generated network logs with Zeek, produced AI classification alerts, stored logs in Elasticsearch, and visualized the data in Kibana. This shows that the proposed architecture can function as a honeypot-based monitoring and detection system.

A key strength of the system is that it combines two levels of observation. Cowrie records the attacker's interaction with the fake system, while Zeek records the related network traffic. This combination provides both behavioral and network-level views of the same activity. The AI detector then uses Zeek's network features to classify traffic behavior.

There are still limitations. First, the system was tested in a controlled local environment rather than on a public network. This was necessary for safety and ethical reasons, but it means the system was not exposed to large volumes of real attacker traffic. Future testing should be

conducted in an approved isolated lab network to capture more realistic traffic.

Second, the live test activity was limited. The SSH session was manually generated and small in scale, so it may not represent the full range of attack patterns in the IoT-23 training data. More varied attack simulations are needed to evaluate the model under different live conditions.

Third, the Random Forest model achieved high accuracy on the prepared IoT-23 dataset, but dataset accuracy does not guarantee the same performance in a live deployment. Real environments may contain traffic patterns that differ from the training data. Further validation using live honeypot traffic and additional attack scenarios is therefore required.

Fourth, Elasticsearch, Logstash, and Kibana consume more resources than the core honeypot and detector components. They provide strong storage and visualization capabilities, but long-term monitoring or higher traffic volume may require memory and performance tuning.

Overall, the results show that TrapPot is a successful proof-of-concept system. It demonstrates how Cowrie, Zeek, Random Forest classification, and the ELK Stack can be integrated in a Docker-based environment for IoT honeypot monitoring and analysis. The system provides a strong foundation for future improvements, including broader attack testing, expanded validation, and more detailed dashboard analytics.

## Chapter 6

# CONCLUSIONS AND FUTURE WORK

## CONCLUSIONS AND FUTURE WORK

This chapter summarizes the main results of the TrapPot project and reflects on the implementation and testing outcomes. It also presents the current limitations and future improvements that can make TrapPot stronger as a cybersecurity research and learning platform.

### 6.1 Discussion of Results

The project results show that TrapPot was successfully deployed as a Docker-based honeypot, monitoring, classification, and visualization system. The implemented environment brings together Cowrie for SSH and Telnet honeypot interaction, Zeek for network traffic monitoring, a Random Forest-based AI detector for classification, and the ELK Stack for log storage and visualization.

During testing, Cowrie accepted SSH connections on port 2222 and provided a fake Linux shell environment. The honeypot logged login attempts, executed commands, and file download attempts. This confirms that the honeypot layer fulfilled its main purpose: capturing attacker-like behavior without exposing a real production system.

Zeek also played an important role. It monitored the traffic generated by the Cowrie session and produced structured logs such as `conn.log` and `ssh.log`. These logs provided network-level information, including ports, protocols, services, duration, byte counts, packet counts, and connection states. This made Zeek the link between honeypot interaction and AI-based traffic classification.

The AI detector read Zeek's `conn.log`, extracted the required twelve network traffic features, and applied the trained Random Forest model. The detector generated live classification alerts and wrote detection records to `detections.json`. This shows that the machine learning component was integrated into the live TrapPot pipeline rather than kept as a separate offline experiment.

The collected data was processed and presented through the ELK Stack. Logstash shipped records to Elasticsearch, and Kibana provided data views and the **TrapPot Overview** dashboard. This allowed the project team to inspect system activity through a browser interface instead of relying only on terminal logs.

Overall, TrapPot achieved its primary goal as a proof-of-concept system. It shows how honeypot technology, network monitoring, machine learning, and visualization tools can be

combined to support IoT attack monitoring and analysis.

### 6.1.1 Contributions

The main contribution of TrapPot is an integrated monitoring workflow that combines attacker-interaction capture, network traffic analysis, machine learning classification, and log visualization. The system connects Cowrie, Zeek, the AI detector, and the ELK Stack inside one Docker-based environment, allowing collected activity to be analyzed from both honeypot and network perspectives.

TrapPot also provides a practical AI detection workflow for the project environment. The Random Forest model is used by the live detector, which reads Zeek's `conn.log`, extracts the twelve required network features, and produces classification alerts at runtime. This demonstrates that the trained model was connected to the deployed system pipeline.

Another contribution is portability. Docker Compose makes it easier to start the system and reproduce it on other machines without manually installing and configuring each component. This makes TrapPot suitable for demonstrations, testing, and future development in a controlled lab environment.

## 6.2 Limitations

Although TrapPot produced promising results, several limitations remain:

- **Controlled Testing Environment:** The system was tested in a local lab environment and was not exposed to public attacker traffic. This was necessary for safety and ethical reasons, but it means the system was not tested against a large volume of real-world attack activity.
- **Dataset-Based Model Accuracy:** The Random Forest model achieved approximately 99.59% accuracy on the prepared IoT-23 dataset. However, high dataset accuracy does not guarantee identical performance in a live deployment, especially when real traffic differs from the training data.
- **Limited Live Attack Variety:** Live testing mainly used manual SSH interaction and simple commands. Additional attack scenarios are needed to evaluate scanning,

brute-force attempts, malware download behavior, and command execution patterns more deeply.

- **Feature Scope:** The current AI detector mainly uses Zeek `conn.log` features. These features are useful for network-based classification, but they do not fully represent detailed Cowrie session behavior such as command sequences, login-attempt patterns, and downloaded file indicators.
- **Resource Requirements:** Elasticsearch, Logstash, and Kibana provide powerful storage and visualization capabilities, but they require more memory and processing power than the honeypot and AI detector components. Long-term deployment or high traffic volume may require performance tuning.

These limitations do not remove the value of the project. Instead, they define the next steps required to move TrapPot from a working prototype toward a stronger research and lab deployment.

## 6.3 Future Work

Several improvements can extend the practical value of TrapPot.

### 6.3.1 Testing with Realistic Attack Traffic

Future versions of TrapPot should be tested in an approved isolated network environment with more realistic traffic. This would allow the system to collect genuine attacker behavior safely. A larger live dataset would also help evaluate how well the AI detector performs under different attack conditions.

Future tests should include repeated login attempts, scanning activity, automated bots, file download attempts, and command execution. The current version was tested manually, so broader validation is needed.

### 6.3.2 Combining Zeek and Cowrie Features

The current AI detector relies on twelve network-level features derived from Zeek's `conn.log`. Future work can improve classification by combining these features with Cowrie session-level

features. Examples include number of login attempts, command rate, command sequence patterns, session length, file download indicators, and suspicious command usage.

Combining Zeek and Cowrie features would allow the model to analyze both network behavior and attacker interaction behavior. This may improve classification quality and provide a more complete view of malicious activity.

### **6.3.3 Indexing AI Results in Elasticsearch**

The current implementation prints live alerts and saves detection records in `detections.json`. A useful improvement is to send these results directly to Elasticsearch or continue strengthening the Logstash-based ingestion path for detector output. This would allow Kibana to show AI predictions, detected categories, and detection timestamps in the same dashboard as Cowrie and Zeek activity.

### **6.3.4 Exploring Additional Machine Learning Models**

The Random Forest model achieved strong results on the prepared IoT-23 dataset, but future work can compare it with other models such as XGBoost, Support Vector Machines, and neural network-based approaches. Comparing models would help determine whether another method can reduce false predictions or handle more complex attack patterns.

### **6.3.5 Improving Deployment Security**

Future iterations can improve deployment security through stricter container isolation, stronger network controls, controlled access to Kibana, backup policies, and clearer log retention rules. These improvements would make TrapPot more suitable for longer controlled lab deployments and reduce the risks associated with running honeypot services.

## **6.4 Conclusion**

TrapPot demonstrates how deception, monitoring, machine learning, and visualization can work together in one controlled cybersecurity environment. The project addresses the challenge of understanding SSH and Telnet-based attacks against IoT-like systems without risking a real production device. It achieved its main goal by providing a working proof of concept for



capturing suspicious interaction, monitoring related network activity, classifying traffic behavior, and presenting the collected information through a dashboard.

Overall, TrapPot provides a strong foundation for cybersecurity education, IoT attack monitoring, and future research development.

## Appendix A

# Project Installation

# References

- [1] M. Antonakakis, T. April, M. Bailey, M. Bernhard, E. Bursztein, J. Cochran, Z. Durumeric, J. A. Halderman, L. Invernizzi, M. Kallitsis, D. Kumar, C. Lever, Z. Ma, J. Mason, D. Menscher, C. Seaman, N. Sullivan, K. Thomas, and Y. Zhou, “Understanding the mirai botnet,” in *26th USENIX Security Symposium (USENIX Security 17)*. Vancouver, BC: USENIX Association, aug 2017, pp. 1093–1110. [Online]. Available: <https://www.usenix.org/conference/usenixsecurity17/technical-sessions/presentation/antonakakis>
- [2] S. Lee, A. Abdullah, N. Jhanjhi, and S. Kok, “Classification of botnet attacks in IoT smart factory using honeypot combined with machine learning,” *PeerJ Computer Science*, vol. 7, p. e350, Jan. 2021. [Online]. Available: <https://doi.org/10.7717/peerj-cs.350>
- [3] A. Famera, B. Hilger, S. Bhunia, and P. Heil, “Analyzing the mirai IoT botnet and its recent variants: Satori, mukashi, moobot, and sonic,” *arXiv preprint arXiv:2508.01909*, August 2025. [Online]. Available: <https://arxiv.org/abs/2508.01909>
- [4] Y. Zhang, M. Liu, W. Shi, and K. Chen, “Identification of ssh honeypots using machine learning algorithms,” in *2023 IEEE International Conference on Cyber Security and Resilience (CSR)*. IEEE, 2023, pp. 482–487. [Online]. Available: <https://ieeexplore.ieee.org/document/10173858>
- [5] A. Mahmood, R. Latif, W. Iqbal, F. Alghamdi, and S. Ahmad, “Exploring security threats and solutions techniques for internet of things IoT networks: A comprehensive review,” *Frontiers in Artificial Intelligence*, vol. 7, p. 1397480, May 2024. [Online]. Available: <https://doi.org/10.3389/frai.2024.1397480>
- [6] G. Suárez-Tangil, F. Lombardi, and D. Rossi, “Stories from a customized honeypot for the IoT,” *Journal of Internet Technology*, vol. 24, no. 4, pp. 827–838, 2023. [Online]. Available: <https://suarez-tangil.networks.imdea.org/papers/2023jit-honeypot.pdf>

- 
- [7] S. Islam, A. Rahman, A. Iqbal, M. Hasan, K. Biswas, S. Saha *et al.*, “A large-scale study of IoT security weaknesses and vulnerabilities in code snippets shared by developers,” *arXiv preprint arXiv:2308.13141*, August 2023. [Online]. Available: <https://doi.org/10.48550/arXiv.2308.13141>
- [8] D. S. Morozov, S. A. Korchagin, and S. T. Gataullin, “The sweet taste of IoT deception: an adaptive honeypot framework for design and evaluation,” *Journal of Edge Computing*, vol. 2, no. 11, November 2024. [Online]. Available: <https://doi.org/10.55056/jec.607>
- [9] D. Commey, S. Hounsinnou, and G. V. Crosby, “Blockchain-enabled dynamic honeypot conversion for enhanced IoT security,” *Journal of Information Security and Applications*, vol. 87, p. 103990, March 2025. [Online]. Available: <https://doi.org/10.1016/j.jisa.2025.104109>
- [10] J. Franco, A. Aris, B. Canberk, and A. S. Uluagac, “A survey of honeypots and honeynets for internet of things, industrial internet of things, and cyber-physical systems,” *IEEE Communications Surveys & Tutorials*, vol. 23, no. 4, pp. 2351–2383, 2021. [Online]. Available: <https://doi.org/10.1109/COMST.2021.3106669>
- [11] H. Hindy, R. Atkinson, C. Tachtatzis, J.-N. Colin, E. Bayne, and X. Bellekens, “A comprehensive survey on machine learning-based intrusion detection systems for secure communication in internet of things,” *Wireless Communications and Mobile Computing*, vol. 2023, p. 8981988, 2023. [Online]. Available: <https://doi.org/10.1155/2023/8981988>
- [12] A. Alzahrani, R. Alharthi, R. Alwafi, M. Aljuhani, and M. Al-Harbi, “Lightweight machine learning techniques for multi-class attack detection in IoT networks using optimized feature engineering,” *arXiv preprint arXiv:2502.04057*, February 2025. [Online]. Available: <https://arxiv.org/abs/2502.04057>
- [13] F. De Keersmaecker, Y. Cao, G. K. Ndonga, and R. Sadre, “A survey of public IoT datasets for network security research,” *IEEE Communications Surveys & Tutorials*, vol. 25, no. 3, pp. 1808–1840, thirdquarter 2023. [Online]. Available: <https://doi.org/10.1109/COMST.2023.3288942>
- [14] R. A. Rodriguez-Gomez, L. F. Maimo, A. L. Perales-Gomez, and F. J. Garcia-Clemente, “A systematic review of honeypot data collection, threat intelligence platforms,
-

- and AI/ML techniques,” *SSRN Electronic Journal*, May 2025. [Online]. Available: <https://doi.org/10.2139/ssrn.5242873>
- [15] C. T. Alex, G. Creado, W. Almobaideen, O. AbuAlghanam, A. A. Margret, A. Fournaris, and K. Markantonakis, “A comprehensive survey for IoT security datasets taxonomy, classification and machine learning mechanisms,” *Computers & Security*, vol. 132, p. 103283, September 2023. [Online]. Available: <https://doi.org/10.1016/j.cose.2023.103283>
- [16] E. Alajrami and S. S. Abu-Naser, “AI-powered intrusion detection system with honeypot integration,” *International Journal of Intelligent Information Systems*, vol. 14, no. 4, pp. 44–58, August 2025. [Online]. Available: <https://doi.org/10.11648/j.ijiis.20251404.11>
- [17] M. Saied, S. Guirguis, and M. Madbouly, “A comparative analysis of using ensemble trees for botnet detection in IoT networks,” *Scientific Reports*, vol. 13, no. 1, p. 21756, December 2023. [Online]. Available: <https://doi.org/10.1038/s41598-023-48681-6>
- [18] E. D. Saputro, Y. Purwanto, and M. F. Ruriawan, “Medium interaction honeypot infrastructure on the internet of things,” in *2020 IEEE International Conference on Internet of Things and Intelligence System (IoTaIS)*, 2021, pp. 98–102. [Online]. Available: <https://doi.org/10.1109/IoTaIS50849.2021.9359711>
- [19] O. Surnin, F. Hussain, R. Hussain, S. Ostrovskaya, A. Polovinkin, J. Lee, and X. Fernando, “Probabilistic estimation of honeypot detection in internet of things environment,” in *2019 International Conference on Computing, Networking and Communications (ICNC)*, Feb 2019, pp. 191–196. [Online]. Available: <https://doi.org/10.1109/ICCNC.2019.8685566>
- [20] S. Lanz, J. M. Pedersen, and E. Vasilomanolakis, “Optimizing internet of things honeypots with machine learning: A review,” *Applied Sciences*, vol. 15, no. 10, p. 5251, May 2025. [Online]. Available: <https://doi.org/10.3390/app15105251>
- [21] I. M. M. Matin and B. Rahardjo, “The use of honeypot in machine learning based on malware detection: A review,” in *2020 8th International Conference on Cyber and IT Service Management (CITSM)*, 2020, pp. 1–6. [Online]. Available: <https://doi.org/10.1109/CITSM50537.2020.9268794>
- [22] W. Cabral, C. Valli, L. Sikos, and S. Wakeling, “Review and analysis of cowrie artefacts and their potential to be used deceptively,” in *2019 International Conference*

- on *Computational Science and Computational Intelligence (CSCI)*, 2019, pp. 166–171. [Online]. Available: <https://doi.org/10.1109/CSCI49370.2019.00035>
- [23] M. Başer, E. Y. Güven, and M. A. Aydın, “SSH and telnet protocols attack analysis using honeypot technique: Analysis of SSH and Telnet honeypot,” in *2021 6th International Conference on Computer Science and Engineering (UBMK)*, 2021, pp. 806–811. [Online]. Available: <https://doi.org/10.1109/UBMK52708.2021.9558948>
- [24] C.-K. Tsung, C.-T. Yang, and S.-W. Yang, “Visualizing potential transportation demand from ETC log analysis using ELK stack,” *IEEE Internet of Things Journal*, vol. 7, no. 7, pp. 6623–6633, 2020. [Online]. Available: <https://doi.org/10.1109/JIOT.2020.2974671>
- [25] J. Franco, A. Aris, B. Canberk, and A. S. Uluagac, “A survey of honeypots and honeynets for internet of things, industrial internet of things, and cyber-physical systems,” *IEEE Communications Surveys & Tutorials*, vol. 23, no. 4, pp. 2351–2383, 2021. [Online]. Available: <https://doi.org/10.1109/COMST.2021.3106669>
- [26] I. Karim, Q.-T. Vien, T. A. Le, and Y. Cao, “An efficient intrusion detection system for IoT security using shapley additive explanations and random forest,” *IEEE Access*, vol. 12, pp. 130 341–130 359, 2024. [Online]. Available: <https://doi.org/10.1109/ACCESS.2024.3454361>
- [27] R. Kalaria, A. S. M. Kayes, W. Mayer, P. P. Jayaraman, and M. M. Gaber, “IoTPredictor: A security framework for predicting IoT device behaviours and detecting malicious devices against cyber attacks,” *Computers & Security*, vol. 146, p. 104037, November 2024. [Online]. Available: <https://doi.org/10.1016/j.cose.2024.104037>
- [28] D. S. Mary, M. S. Mahilan, and K. S. Kumar, “Network intrusion detection: An optimized deep learning approach using big data analytics,” *Expert Systems with Applications*, vol. 237, p. 121744, February 2024. [Online]. Available: <https://doi.org/10.1016/j.eswa.2023.121744>
- [29] A. Albaseer, A. Alomari, M. Abdallah, and A. Al-Fuqaha, “FedPot: A quality-aware collaborative and incentivized honeypot-based federated learning for smart grid attack detection,” *IEEE Transactions on Network and Service Management*, vol. 21, no. 4, pp. 4844–4860, August 2024. [Online]. Available: <https://doi.org/10.1109/TNSM.2024.3387710>

- 
- [30] Y. Huang and E. Bertino, “Log2Evt: Constructing high-level events for IoT systems through log analysis,” *Journal of Information Security and Applications*, vol. 87, p. 103850, February 2025. [Online]. Available: <https://doi.org/10.1016/j.jisa.2024.103850>
- [31] M. Di Mauro, G. Galatro, G. Fortino, and A. Liotta, “Supervised feature selection techniques in network intrusion detection: A critical review,” *Engineering Applications of Artificial Intelligence*, vol. 101, p. 104216, May 2021. [Online]. Available: <https://doi.org/10.1016/j.engappai.2021.104216>
- [32] M. M. Ahsan, S. A. Luna, and Z. Siddique, “Effects of feature selection and normalization on network intrusion detection,” *Information Security Journal: A Global Perspective*, vol. 33, no. 2, pp. 145–162, 2024. [Online]. Available: <https://www.sciencedirect.com/science/article/pii/S2666764924000390>
- [33] N. Khan, A. Abdullah, and A. S. Shah, “Optimization of predictive performance of intrusion detection system using confusion matrix and f1-score,” *PeerJ Computer Science*, vol. 9, p. e1499, September 2023. [Online]. Available: <https://doi.org/10.7717/peerj-cs.1499>
- [34] S. Garcia, A. Parmisano, and M. J. Erquiaga, “IoT-23: A labeled dataset with malicious and benign IoT network traffic,” Zenodo dataset, 2020, accessed 2026-06-01. [Online]. Available: <https://www.stratosphereips.org/datasets-iot23>
- [35] J. I. Christy Eunaicy, C. Jayapratha, and H. Salome, “IoT guardian: A novel feature discovery and cooperative game theory empowered feature selection with ML model for IoT threats detection,” *International Journal of Computer Networks and Communications*, vol. 16, no. 2, pp. 1–24, April 2024. [Online]. Available: <https://doi.org/10.5121/ijcnc.2024.16201>
- [36] K. Hopf and S. Reifenrath, “Filter methods for feature selection in supervised machine learning applications: Review and benchmark,” *arXiv preprint arXiv:2111.12140*, November 2021. [Online]. Available: <https://arxiv.org/abs/2111.12140>
- [37] F. Ahmed and M. Rahman, “Optimized IoT intrusion detection using machine learning technique,” *arXiv preprint arXiv:2412.02845*, December 2024. [Online]. Available: <https://arxiv.org/abs/2412.02845>

- 
- [38] H. Pan, Z. Li, X. Wang, R. Gong, H. Li, and X. Zheng, “A complete scheme for multi-character classification using XGBoost with grid search hyperparameter tuning,” *IEEE Transactions on Systems, Man, and Cybernetics: Systems*, vol. 54, no. 5, pp. 2945–2955, May 2024. [Online]. Available: <https://ieeexplore.ieee.org/abstract/document/10470417/>
- [39] A. Alfrhan and T. Alhusain, “SMOTE: Class imbalance problem in intrusion detection system,” in *2020 3rd International Conference on Computer Applications & Information Security (ICCAIS)*, 2020, pp. 1–8. [Online]. Available: <https://ieeexplore.ieee.org/abstract/document/9213728>
- [40] M. J. F. Alenazi, S. Hina, and H. M. Alshahrani, “SMOTE-stack for network intrusion detection in an IoT environment,” in *2022 14th International Conference on Computational Intelligence and Communication Networks (CICN)*, 2022, pp. 334–339. [Online]. Available: <https://ieeexplore.ieee.org/abstract/document/10427217>
- [41] Y. S. Abdulsalam and M. Hedabou, “Multi-class network intrusion detection with class imbalance via SMOTE and focal loss,” *arXiv preprint arXiv:2310.01850*, October 2023. [Online]. Available: <https://arxiv.org/abs/2310.01850>
- [42] M. Alawadi, M. Alshammari, and S. Al-Hejaili, “A comprehensive study on IoT honeypots: Models, taxonomies, tools, and future directions,” *IEEE Internet of Things Journal*, vol. 9, no. 18, pp. 16 983–17 000, September 2022. [Online]. Available: <https://doi.org/10.1109/COMST.2021.3106669>
- [43] W. Fan, Z. Du, M. Smith-Creasey, and D. Fernández, “HoneyDOC: An efficient honeypot architecture enabling all-round design,” *IEEE Journal on Selected Areas in Communications*, vol. 37, no. 3, pp. 683–697, March 2019. [Online]. Available: <https://doi.org/10.1109/JSAC.2019.2894307>
- [44] A. A. Wardana, M. Turnip, and R. F. Sari, “Federated random forest with feature selection for intrusion detection system on internet of things,” *Procedia Computer Science*, vol. 234, pp. 1075–1084, 2024. [Online]. Available: <https://doi.org/10.1016/j.procs.2024.03.075>